

Versión traducida por Alejandra Serrano contactar a:
contacto@reparaciondepc.cl

No existen balas de plata: esencias y accidentes de la ingeniería software

¿Tiene que ser difícil? Dificultades esenciales

No solo ahora no hay balas de plata a la vista, sino que la propia naturaleza del software hace poco probable su futura existencia; ninguna futura invención servirá para la productividad del software, confianza, y simplicidad, como lo hicieron sistemas electrónicos, transistores, y la integración a larga escala por los hardware de computadores. No podríamos esperar jamás ver duplicarse las ganancias cada 2 años.

En primer lugar, se debe observar que la anomalía no es que el progreso del software sea lento, sino que el progreso de los hardware de computadores es muy rápido. Ninguna otra tecnología desde el principio de la civilización ha visto seis veces elevado a la décima potencia su ganancia en cuanto al precio de presentación en los últimos 30 años. En ningún otro tipo de tecnología se puede optar por tomar, ya sea en la ganancia o la mejora de los resultados en la reducción de los costos. Esas ganancias provienen de la transformación de la fabricación del computador de una industria del lenguaje Assembly a una industria de proceso.

En segundo lugar, para ver el nivel de progreso que se puede esperar en la tecnología software, permítanos examinar las dificultades de esa tecnología. Siguiendo a Aristóteles, divido en "esencia" las dificultades inherentes a la naturaleza del software, y "accidentes" a las dificultades que actualmente asisten a su producción pero que no son inherentes.

La esencia de una entidad software es el resultado de una construcción de conceptos entrelazados: conjuntos de datos, relaciones entre los elementos de datos, algoritmos y de invocaciones de funciones. Esta esencia es abstracta, ya que tal construcción conceptual es la misma bajo muchas representaciones distintas. Sin embargo es extremadamente preciso y detallado.

Creo q la parte más difícil en la creación de un software es la especificación, diseño y prueba de la construcción de esta base conceptual, no el trabajo de representar y probar la calidad de la representación. Aun cometemos errores de sintaxis para asegurarnos, pero no son nada comparados con los errores conceptuales en la mayoría de los sistemas.

Si esto es cierto, crear un software será siempre difícil. No hay intrínsecamente ninguna bala de plata.

Consideremos las propiedades inherentes de esta esencia irreducible de los sistemas modernos de software: complejidad, conformidad, variabilidad e invisibilidad.

Complejidad: Las entidades software son más complejas que talvez cualquier otra construcción humana por su tamaño, porque no tiene dos partes iguales (al menos por encima del nivel del orden). Y si las hay, ponemos las dos partes similares en una subrutina; abierta o cerrada. En este aspecto, los sistemas software difieren profundamente de los computadores, edificios o automóviles, donde abundan elementos repetidos.

Los computadores digitales son por si mismos más complejos que la mayoría de las cosas que las personas crean: tienen un gran número de etapas. Esto hace difícil concebirlos, describirlos y probarlos. Los sistemas software tienen muchísimos más estados que los computadores.

Del mismo modo, un aumento de una entidad software no es simplemente una repetición de los mismos elementos a una mayor escala, es necesario un aumento en el número de distintos elementos. En la mayoría de los casos, los elementos interactúan entre sí de una forma no lineal, y la complejidad del todo incrementa mucho más que linealmente.

La complejidad del software es una propiedad esencial, no accidental. Por lo tanto, las descripciones de una identidad software que dejan de lado su complejidad también lo hacen de su esencia. Durante tres siglos, las matemáticas y las ciencias físicas hicieron grandes avances construyendo modelos simplificados de fenómenos complejos, derivando propiedades de los modelos y comprobando esas derivaciones por medio de experimentación. Este paradigma funcionaba porque las complejidades ignoradas en esos modelos no eran propiedades esenciales de los fenómenos. No funciona cuando las complejidades son la esencia.

Muchos de los típicos problemas de desarrollar productos software provienen de esta complejidad esencial y su no-linealidad incremental con el tamaño. De la complejidad proviene la dificultad de comunicación entre los miembros del equipo, lo que lleva a fallas de producto, costos excesivos, retrasos en los periodos de entrega. De la complejidad proviene la dificultad de enumeración, un aún menor entendimiento y todos los posibles estados del programa; de allí viene la poca fiabilidad. De la complejidad de función viene la dificultad de iniciar la función, lo cual hace a los programas difíciles de usar. De la complejidad de estructura viene la dificultad para extender los programas a nuevas funciones sin crear efectos secundarios. De la complejidad de estructura vienen los estados invisibles, los cuales constituyen una verdadera trampa de seguridad.

De la complejidad no solo vienen problemas técnicos, sino también problemas de manejo. Esto hace difícil una visión general, impidiendo así una integridad conceptual. Hace difícil encontrar y controlar todos los cabos sueltos. Crea la tremenda carga de aprender y entender, lo que hace de la renovación de personal un desastre.

Conformidad: La gente de software no es la única en enfrentar complejidad. Físicos tratan con objetos extremadamente complejos, incluso al nivel de partículas fundamentales. Sin embargo, la labor de un físico es trabajar con la firme convicción de estar unificando principios aún no descubiertos, ya sea en quarks o en "teorías del campo unificado". Einstein sostuvo que debe haber explicaciones simplificadas de la naturaleza, porque Dios no es caprichoso ni arbitrario.

Esta creencia no reconforta al ingeniero de software. Gran parte de la complejidad que él debe dominar es complejidad arbitraria, forzada sin ton ni son por las muchas instituciones humanas y sistemas a los cuales sus interfaces deben conformar. Estos difieren de interfaz a interfaz y de vez en cuando no es por necesidad, sino solo por que fueron diseñados por diferentes personas, en vez de por Dios.

En muchos casos el software debe ajustarse porque es el más reciente. Otras veces debe adaptarse porque es visto como el más mediocre. Pero en todos los casos, gran complejidad viene de la conformación de otras interfaces; esta complejidad no puede simplificarse por ningún rediseño aislado del software.

Mutabilidad: La entidad software está constantemente sujeta a presiones de cambio. Por supuesto también lo están los edificios, autos y computadores. Pero las cosas fabricadas son cambiadas con poca frecuencia luego de su creación, son reemplazadas por modelos nuevos, o se incorporan cambios esenciales en el número de serie de copias posteriores al diseño básico. Retrocesos en automóviles son muy poco frecuentes, e incluso se dan menos en el campo computacional.

Ambos son mucho menos frecuentes que las modificaciones en el terreno de software.

En parte esto es así porque el software de un sistema abarca su función, y la función es la parte que más siente la presión de cambio. Y esto es en parte es porque el software puede ser cambiado con mayor facilidad, **it is pure thought-stuff**, infinitamente adaptable. Los edificios son cambiados, pero los altos costos de estos cambios son suficientes para desalentar a quienes quieran cambiarlos. **Todo software exitoso es cambiado.** Se trabajan dos procesos. Primero, cuando un producto software es útil, las personas lo prueban llevándolo al límite o más allá de su dominio original. La presión para extender las funciones viene principalmente de usuarios a los que les gustan las funciones básicas y les inventan nuevos usos.

Segundo, **un software exitoso sobrevive mas allá de la vida normal de máquina para la que fue creada.** Si no nuevos computadores, al menos vendrán nuevos discos, pantallas e impresoras, y el software debe ajustarse.

En resumen, el producto software esta incrustado en una matriz cultural de aplicaciones, usuarios, leyes y máquinas vehículo. Todos estos cambian continuamente, lo que fuerza implacablemente cambios en el producto software.

Invisibilidad: Software es invisible. Las abstracciones geométricas son herramientas poderosas. El primer piso de un edificio ayuda a cliente y arquitecto a evaluar los espacios, flujos de trafico y vistas. Las contradicciones y omisiones se vuelven evidentes. Dibujos a escala de partes mecánicas y maquetas, a pesar de las abstracciones, tienen la misma finalidad. Una realidad geométrica es capturada en una abstracción geométrica.

La realidad del software no esta intrínsecamente incrustada en el espacio, por lo tanto, no tiene aun una representación geométrica de la forma en que la Tierra tiene mapas, los chips de silicio tienen diagramas, los computadores tienen diagramas de conectividad. Tan pronto como intentamos esquematizar la estructura del software, nos encontramos con que constituye no uno, sino varios gráficos generales superpuestos uno sobre el otro. Los varios gráficos pueden representar el flujo de control, el flujo de datos, los patrones de dependencia, la secuencia de tiempo y las relaciones nombre-espacio. Estos gráficos por lo general no son siquiera planos, y mucho menos jerárquicos. De hecho, una de las formas de establecer un control conceptual sobre tal estructura es imponer un corte de la conexión hasta que uno o mas gráficos se convierta en jerárquico.

A pesar del progreso en la limitación y simplificación de las estructuras del software, estas siguen siendo intrínsecamente invisibles y por lo tanto no le permiten a la mente usar algunas de sus más poderosas herramientas conceptuales. Esta carencia no solo le impide el proceso de diseño a una sola mente, sino que también dificulta gravemente la comunicación entre mentes.

Últimos Avances Resolvieron Dificultades Accidentales

Si analizamos los tres pasos en el desarrollo de la tecnología software que han sido más fructíferos en el pasado, descubrimos que cada uno atacó a una de las dificultades principales en la construcción de software, pero que esas dificultades habían sido accidentales y no esenciales. También podemos ver los límites naturales a la extrapolación de cada uno de dichos ataques.

Lenguajes de alto nivel: Sin duda el golpe más poderoso para la productividad, fiabilidad y simplicidad del software ha sido la utilización progresiva de lenguajes de alto nivel para la programación. La mayoría de los observadores le acreditan el

desarrollo de al menos un factor de cinco en cuanto a productividad, y con un aumento simultáneo de la fiabilidad, simplicidad y comprensibilidad.

¿Qué logra un lenguaje de alto nivel? Libera a un programa de gran parte de su complejidad accidental. Un programa abstracto está formado por construcciones conceptuales: operaciones, tipos de datos, secuencias y comunicación. El programa concreto involucra bits, registros, divisiones, canales, discos y demases. En la medida en que el alto nivel de lenguaje incorpora las construcciones que uno quiera en el programa abstracto y evite todos los de menor nivel, éste elimina todo un nivel de complejidad que nunca fue inherente al programa en absoluto.

Lo máximo que un lenguaje de alto nivel puede hacer es presentar todas las construcciones que el programador imagina en el programa abstracto. Sin duda, el nivel de nuestras ideas acerca de las estructuras de datos, tipos de datos y operaciones está creciendo, pero en una tasa siempre decreciente. Y el desarrollo del lenguaje se acerca cada vez más a la complejidad de los usuarios.

Tiempo compartido (multiprogramación): El tiempo compartido significa una gran mejora en la productividad de los programadores y en la calidad de sus productos, aunque no tanto así como la que trajo el lenguaje de alto nivel.

El tiempo compartido ataca una dificultad bastante diferente. Este preserva la inmediatez, y por lo tanto permite que se mantenga una visión general de la complejidad. La lenta respuesta de los lotes de programación significa inevitablemente que uno se olvide de detalles minúsculos, si no la propia idea de lo que se pensaba cuando se detuvo la programación y se pidió la compilación y ejecución. Esta interrupción es costosa en tiempo, ya que uno debe refrescarse la memoria. El efecto más grave puede ser la dificultad para comprender todo lo que está sucediendo en un sistema complejo.

Una lenta rotación, como las complejidades lenguaje-máquina, es una dificultad accidental del proceso de software en vez de esencial. Los límites de la contribución potencial de tiempo compartido se derivan directamente. El efecto principal de la utilización compartida es para acortar el tiempo de respuesta del sistema. Cuando el tiempo de respuesta llega a cero, de alguna forma traspasa el umbral de humano de lo observable, unos 100 milisegundos. Más allá de ese umbral no se esperan beneficios.

Entornos de programación unificados: "Unix and Interlisp", el primer entorno de programación integrado en volverse masivo parece haber mejorado su productividad por factores integrales. ¿Por qué?

Ellos atacan las dificultades accidentales que derivan de la utilización de programas individuales juntos, mediante el suministro de bibliotecas integradas, archivos de formato unificado y de tuberías y filtros.

Como resultado de esto, las estructuras conceptuales que básicamente podrían siempre llamar, suministrar datos y usarse entre si pueden, de hecho, hacerlo fácil en la práctica.

Este avance a su vez estimuló el desarrollo de whole toolbenches, ya que cada nueva herramienta podría aplicarse a cualquier programa que utilice el formato estándar.

Debido a estos éxitos, los entornos son objeto de parte importante de las actuales investigaciones de ingeniería software. Veremos lo que promete y sus limitaciones en la siguiente sección.

Esperanzas de la Plata

Ahora consideremos los desarrollos técnicos que son considerados como posibles "balas de plata" con mas frecuencia. ¿Qué problemas enfrentan, de esencia o de las dificultades accidentales que permanecen? ¿Ofrecen avances vanguardistas, o en crecientes?

Ada y otros avances de lenguaje de alto nivel. Uno de los desarrollos recientes mas anunciados es Ada, un lenguaje de los años 80 de alto nivel y con propósitos generales. Ada no solo refleja mejoras evolutivas en conceptos de lenguajes, sino que de hecho

Tal vez la filosofía de Ada es más de un anticipo que el lenguaje Ada, ya que es la filosofía de la modularización, de los tipos abstractos de datos, de jerarquización. Ada es excesivamente sustancioso, un resultado natural del proceso por el que se establecen los requisitos de su diseño. Eso no es fatal, ya que el trabajo d estos vocabularios subordinados puede resolver el problema de aprendizaje. Los vocabularios de trabajo pueden resolver el problema de aprendizaje, y los avances en hardware nos significaran MIPS más baratos para pagar los costos de compilación. Avanzar en la estructuración de sistemas de software es, en efecto, un buen aprovechamiento para los incrementados MIPS que nuestros dólares comprarán. Los sistemas operativos, denunciados a toda voz en la década del 60 por su memoria y ciclo de sus costos, ha demostrado ser una excelente forma en la cual usar algunos de los MIPS y bites de memoria baratos, del pasado surgimiento de hardware.

No obstante, puede que Ada no resulte ser la bala de plata que mata al monstruo de productividad de software. Es, después de todo, sólo otro lenguaje de alto nivel, y el mayor rendimiento de estas lenguas provienen de la primera transición; la transición de las complejidades accidentales en la maquina al orden mas abstracto de las soluciones "paso a paso". Una vez que esos accidentes han sido eliminados, los restantes serán menores, y el costo de su remoción será menor. Auguro que dentro de una década a partir de ahora, cuando la eficacia de Ada sea evaluada, se verá que han hecho una diferencia sustancial, pero no por alguna característica particular del lenguaje, ni siquiera por todas ellas en conjunto. Tampoco los nuevos entornos de Ada probaran ser la causa de su mejora. La contribución más grande de Ada será que utilizarlo significara la capacitación de los programadores a modernas técnicas de diseño de software.

Programación orientada a objetos: Muchos estudiantes de la técnica tienen mas expectativas en cuanto a programación orientada a objetos que cualquier otra técnica de moda. Yo estoy entre ellos. Mark Sherman de Darmouth nota en CSnet News que uno debe ser cuidadoso para poder distinguir dos ideas separadas que están bajo el titulo "tipos abstractos de datos y tipos jerárquicos". El concepto de los tipos de dato abstractos se refiere a que un tipo de objeto debe ser definido con un nombre, un conjunto de valores y un conjunto adecuado de operaciones y no por su estructura de almacenamiento, la cual debe ser ocultada. Ejemplos de ello son los paquetes Ada (con clases privadas) y los módulos de Modula.

Los tipos jerárquicos, como la clase Simula-67, le permite a uno definir interfaces generales que pueden ser mejoradas mas tarde administrándoles los tipos de subordinación. Los dos conceptos son diagonales; uno puede estar jerarquizado sin disimulo o disimulo sin jerarquización. Ambos conceptos representan verdaderos avances en el arte de crear un software.

Cada uno remueve otra dificultad accidental del proceso, permitiéndole al diseñador expresar la esencia del diseño sin tener que expresar una gran cantidad de material sintáctico cuyo contenido no añade información. Para ambos, tipos abstractos y jerárquicos, el resultado es la eliminación de una gran cantidad de dificultades accidentales y la posibilidad de un gran numero de expresiones de diseño.

No obstante, estos avances no pueden hacer más que eliminar las dificultades accidentales de la expresión de diseño. La complejidad del diseño en sí es fundamental, y esos ataques no hacen cambio alguno en eso. Se puede obtener una enorme ganancia de la programación orientada a objetos sólo si tipo y especificación innecesaria en nuestro lenguaje de programación es de nueve décimas del trabajo involucrado en el diseño de un producto de programación. Lo dudo.

Inteligencia artificial: Muchas personas esperan el desarrollo en la inteligencia artificial que proporcione el avance revolucionario que significara enormes ganancias en cuanto a productividad y calidad del software. Yo no. Para saber por qué debemos analizar lo que se entiende por "inteligencia artificial".

DL. Parnas ha aclarado el caos terminológico.

Dos definiciones muy distintas de IA son comúnmente usadas hoy en día. IA – 1: El uso de computadores para resolver problemas que anteriormente sólo podían ser resueltos mediante la aplicación de la inteligencia humana. IA – 2: El uso de un conjunto específico de técnicas de programación conocida como heurística o programación basada en las normas. En este enfoque se utilizan humanos expertos para determinar las heurísticas o reglas básicas que usan para resolver problemas... El programa es diseñado para resolver un problema de la forma en que los seres humanos parecen hacerlo.

Algo puede tener cabida en la definición de AI - 1 de hoy, pero una vez que vemos cómo funciona el programa y comprendemos el problema, no vamos a seguir pensando en él como AI... Lamentablemente no puedo identificar un contenido tecnológico que es único en esta área... La mayor parte del trabajo es de problemas específicos, y se necesita algo de abstracción y creatividad para ver como transferirlos.

Yo concuerdo absolutamente con esta crítica. Las técnicas empleadas para el reconocimiento de voz parecen tener poco en común con las que se utilizan para el reconocimiento de imagen, y ambas son diferentes de las utilizadas en sistemas expertos. Me cuesta trabajo ver cómo el reconocimiento de imágenes, por ejemplo, tendrá alguna diferencia notoria en la practica de programación. El mismo problema ocurre con el reconocimiento de voz. Lo difícil en cuanto a la creación de un software decidir qué se quiere decir, no decirlo. Ninguna forma de facilitación de expresiones puede dar mas que ganancias insignificantes.

Los sistemas expertos de tecnología IA-2 merecen una sección propia.

Sistemas expertos: La parte mas avanzada y que se aplica más ampliamente de la inteligencia artificial es la tecnología para crear sistemas expertos. Muchos científicos de software trabajan duro para aplicar esta tecnología al entorno de la creación de un software. ¿Cuál es el concepto y cuales son las perspectivas?

Un "sistema experto" es un programa que contiene un motor de inferencia generalizado y una norma de base, toma de entrada de datos y asunciones, explora las interferencias que derivan de la norma de base, procura conclusiones y consejos, y ofrece explicar los resultados mostrando su razonamiento al usuario. Los motores de inferencia normalmente pueden hacer frente a los datos borrosos o datos probabilísticas y normas, además de la lógica puramente determinista.

Tales sistemas ofrecen ventajas claras sobre los algoritmos diseñados para llegar a las mismas soluciones de los mismos problemas:

- La tecnología motor de la inferencia es desarrollada en una forma independiente de aplicaciones, y luego se aplica a muchos usos. Uno puede justificar un gran esfuerzo en la inferencia de los motores. De hecho, es una tecnología muy avanzada.
- Las partes variables de los materiales característicos de la aplicación están codificados en la norma de base de forma uniforme y se proporcionan herramientas para el desarrollo, la evolución, la prueba y la documentación de la norma de base. Esto regulariza gran parte de la complejidad de la aplicación misma.

El poder de tales sistemas no proviene de mecanismos de inferencia lujosos, sino más bien de mecanismos con bases de conocimiento cada vez más sustanciosas que reflejan de manera más precisa el mundo real. Creo que el avance más importante ofrecido por la tecnología es la separación entre la complejidad y programa en sí.

¿Cómo puede esta tecnología ser aplicada a la tarea de ingeniería de software? De muchas formas: Estos sistemas pueden sugerir normas de interfaz, asesoramiento sobre estrategias de experimentación, recordatorios de errores frecuentes, y ofrecer también sugerencias de optimización.

Considere la posibilidad de un asesor imaginario de prueba, por ejemplo. En su forma más rudimentaria, podríamos decir que sólo enumera sugerencias en cuanto a las posibles causas de dificultades. Entre más estructuras del sistema consagren la base de normas, y ésta tome en cuenta más sofisticadamente los problemas sintomáticos informados, el consejero de prueba se vuelve más y más preciso en las respuestas que genera y las pruebas que recomienda. Un sistema así de especializado se diferencia radicalmente de los sistemas convencionales en que su base de normas debería probablemente ser jerárquicamente modularizada, como lo son los productos software, para que mientras que el producto es modularmente modificado, también lo sea el diagnóstico de base de normas.

El trabajo necesario para generar las normas de diagnóstico es el que habría que hacer de todos modos al generar el conjunto de casos de prueba para los módulos y para el sistema. Si se hace de una manera adecuada, con una estructura uniforme para las normas y un buen motor de inferencia disponible, puede realmente reducir el total de la generación de mano de obra que significan los casos de prueba, y contribuir así a un mantenimiento de por vida y la modificación de pruebas. De la misma manera, uno puede postular otros asesores, probablemente muchos y de manera simple, para las demás partes de la tarea de construcción de software.

A quien desarrolla un programa se le presentan muchas dificultades en el camino por una pronta realización de un útil sistema consejero experto. Una parte fundamental de nuestro imaginario escenario es el desarrollo de formas fáciles de obtener desde la especificación de programas-estructura a la generación automática o semiautomática de las reglas de diagnóstico. Aún más difícil e importante la doble tarea de adquisición de conocimientos: la búsqueda de expertos elocuentes, auto analíticos, que conozcan la razón por la que hacen las cosas, desarrollando técnicas eficientes para la extracción de lo que saben y separando en sus componentes las bases de normas. El prerequisite esencial para la construcción de un sistema experto es disponer de un experto.

La contribución más poderosa de los sistemas expertos, sin duda será poner al servicio de programadores novatos la experiencia y la sabiduría acumulada de los mejores programadores. Esta no es una contribución pequeña. La brecha entre las mejores prácticas de ingeniería de software y las prácticas promedio es inmensa,

talvez la mas grande entre las disciplinas de la ingeniería. Una herramienta que difunde las buenas prácticas sería importante.

Programación "Automática": Por casi 40 años la gente ha estado anticipando y escribiendo acerca de la "programación automática" o sobre la generación de programas resolviendo problemas del orden de la especificación de problemas. Algunos actualmente escriben como si esperaran que esta tecnología proporcione el próximo gran avance.

Parmas sugiere que el término es usado para el glamour, no por su contenido semántico, afirmando:

En resumen, la programación automática siempre ha sido un eufemismo de la programación con un mayor nivel de lenguaje disponible actualmente para el programador.

El argumenta, en esencia, que en la mayoría de los casos tiene que ser dada la explicación del método de solución y no del problema en sí.

Se pueden encontrar excepciones. La técnica de creación de los generadores es muy potente, y es habitualmente utilizada para traer grandes beneficios en la clasificación de los programas. Algunos sistemas para la integración de ecuaciones diferenciales también han permitido la especificación directa del problema, y los sistemas han evaluado los parámetros, elegidos de una biblioteca de métodos de solución, y los programas generados.

Estas aplicaciones tienen propiedades muy favorables:

- Los problemas son fácilmente caracterizados por un número relativamente reducido de parámetros.
- Hay muchos métodos conocidos de solución para proporcionar una biblioteca de alternativas.
- Amplio análisis ha dado lugar a normas explícitas para la selección de técnicas de solución, dados los parámetros del problema.

Es difícil ver como tales técnicas generalizan a un mundo más amplio el sistema software común, donde los casos con propiedades tan impecables son la excepción. Es difícil incluso imaginar como este avance en la generalización puede pasar.

Grafica de la programación: Un tema favorito para la tesis de doctorado en ingeniería de software es programación gráfica o visual, la aplicación de desde gráficos de computador a diseño de software. ^{6,7} Algunas veces la promesa mantenida por ese enfoque es postulada como una analogía con diseño de circuito integrado VLSI, en el cual gráficos computacionales desempeñan un papel tan provechoso. A veces el teórico justifica la metodología teniendo en cuenta los diagramas de flujo como el ideal programa-diseño y suministrando poderosos planteles para construirlos.

Nada ha surgido de tales esfuerzos que resulte convincente, mucho menos emocionante. Estoy convencido de que nada lo hará.

En primer lugar, como he argumentado en otra parte, el diagrama de flujo es una abstracción muy mala de la estructura software. De hecho, es mejor visto el intento de Burks, Von Neumann y Goldstine por proporcionar un control de lenguaje de alto nivel tan necesitado para sus propuestas de computador. De la forma lamentable en que el diagrama de flujo ha sido elaborado actualmente, ha demostrado que no

sirve como herramienta de diseño, los programadores dibujan diagramas de flujo después, y no antes, de describir los programas.

En segundo lugar, las pantallas de hoy en día son demasiado pequeñas en píxeles para mostrar tanto el alcance como la resolución de cualquier diagrama software verdaderamente detallado. La llamada "metáfora de escritorio" de la actual lugar de trabajo es una metáfora "asiento de avión". Cualquier persona a la que se le hayan desordenado papeles mientras esta en el asiento del medio de un avión entenderá, uno puede ver solo unas pocas cosas a la vez. El verdadero escritorio ofrece una visión general y el acceso al azar a una gran cantidad de páginas. Además, cuando llegan ataques de creatividad, se sabe que más de un programador o escritor ha abandonado el escritorio para estar en algún lugar mas espacioso. La tecnología hardware tendrá que desarrollarse considerablemente antes de que el alcance de nuestras extensiones sea suficiente para el diseño de escritorio de software.

Esencialmente, como he argumentado antes, el software es muy difícil de visualizar. Ya sea por los diagramas de control de flujo, referencias variables cruzadas, flujo de datos, estructuras jerárquicas de datos, o lo que sea, uno siente tan solo una dimensión del intrincadamente elaborado software. Si uno fuerza todos los diagramas generados por las muchas visiones relevantes resulta difícil extraer una visión general. La analogía VLSI es esencialmente engañosa, un diseño de circuito integrado es una descripción bidimensional cuya geometría refleja su realización en un espacio tridimensional. Un sistema software no lo es.

Verificación de programa: Gran parte de los esfuerzos en la programación moderna van dirigidos a la prueba y reparación de fallos. ¿Se podrá encontrar talvez una bala de plata luego de eliminar los errores en el origen o en la fase de diseño del sistema? ¿Pueden la productividad y la fiabilidad del producto ser radicalmente mejorados, siguiendo así la estrategia tan opuesta de proveer diseños sin errores, antes de derrochar el inmenso esfuerzo en implementarlos y probarlos?

No creo que la productividad vaya a encontrar magia aquí. La verificación de programa es un concepto muy poderoso y será muy importante para cosas como un seguro manejo de núcleos de sistema (o kernels). Sin embargo esta tecnología no promete ahorrar mano de obra. Las comprobaciones significan tanto trabajo que tan solo unos pocos programas importantes han sido verificados alguna vez. Verificación de programas no significa que estos sean a prueba de errores. Tampoco hay magia aquí. Las pruebas matemáticas también pueden ser fallidas. Así que aunque la verificación podría reducir la carga de lo que significa probar los programas, no la puede eliminar.

Mas grave aún, incluso la verificación de programa perfecta solo puede establecer que un programa cumple su **especificación**. La parte más difícil de la tarea del software es alcanzar una especificación completa y consistente, y gran parte de la esencia de la creación de un programa de hecho es la depuración de errores en la especificación.

Entornos y herramientas: ¿Cuántas ganancias se pueden esperar de la explotación de investigaciones dirigidas hacia mejores entornos de programación? La primera reacción instintiva de uno, es que los grandes problemas de rentabilidad fueron los primeros en ser atacados y han sido resueltos; sistemas jerárquicos de archivos, archivos de formato uniforme para hacer posible interfaces uniformes de programa, y herramientas generales. Los editores inteligentes con lenguaje específico son avances que aun no se utilizan masivamente en la práctica, pero lo máximo que pueden prometer es no tener errores sintácticos ni errores semánticas simples.

Talvez el mayor logro de los entornos de programación será el uso de bases de datos integradas para seguirles el rastro al gran número de detalles que deben ser recordados con precisión por el programador, y mantenido al día por un grupo de colaboradores en un solo sistema.

Ciertamente este trabajo vale la pena, y sin duda dará frutos en cuanto a productividad y confiabilidad, pero por su propia naturaleza los beneficios serán insignificantes.

Estaciones de trabajo: ¿Qué ganancias puede esperar el material gráfico de software del incremento seguro y veloz de la capacidad de memoria y poder de la estación de trabajo individual? ¿Cuántos MIPS pueden ser utilizados fructíferamente? La composición y edición de programas y documentos es totalmente respaldada por **today's speeds. Compiling** puede significar un aumento, pero un factor de cada 10 en velocidad de máquina **would surely leave thinktime** a la actividad dominante en el día del programador. De hecho parece serlo ahora.

Ciertamente le damos la bienvenida a estaciones de trabajo más poderosas. No podemos esperar mejoras mágicas de ellas.

Ataques prometedores en la esencia conceptual

Aunque ningún avance tecnológico promete una clase de resultados mágicos como a los que estamos acostumbrados en el área del hardware, hay una abundancia de buen trabajo ahora y la promesa de un progreso permanente.

Todos los ataques tecnológicos a los accidentes del proceso software están fundamentalmente limitados por la ecuación de productividad:

Tiempo de tarea = $\sum (\text{frecuencia})_i \times (\text{tiempo})$.

Si, como se cree, los componentes conceptuales de la tarea están tomando la mayor parte del tiempo, entonces ninguna cantidad de actividad en los componentes de la tarea, que son solamente la expresión de conceptos, puede brindar ganancias considerables.

Por lo tanto debemos considerar esos ataques dirigidos a la esencia del problema software, la formulación de esas complejas estructuras conceptuales. Afortunadamente algunos de estos ataques son prometedores.

Comprar versus crear: La solución más radical y posible para la construcción de un software es no construirlo.

Cada día esto es más fácil, más y más vendedores ofrecen más y mejores productos software para una variedad de aplicaciones. Mientras que nosotros, ingenieros software, hemos trabajado en la metodología de producción, la revolución de los computadores personales ha creado no uno, sino muchos mercados masivos para software. Cada quiosco trae mensualmente revistas, los cuales promocionan docenas de productos a precios que van desde unos pocos dólares a unos cuantos cientos de dólares. Fuentes mas especializadas ofrecen productos mas poderosos para la estación de trabajo y otros mercados "Unix". Incluso herramientas software y entornos pueden ser traídos listos para su

utilización. Yo he propuesto en todas partes un mercado para los módulos individuales.

Cualquiera de estos productos es más barato que construir uno nuevo. Incluso a un costo de cien mil millones de dólares, un software comprado cuesta solo cerca de lo que cuesta un programador por un año. ¡Y la entrega es inmediata! Al menos los productos que realmente existen, productos cuyo creador puede dirigir a un usuario feliz. Mas aun, tales productos tienden a ser mucho mejor certificados y de alguna forma mejor mantenidos que los **de creación personal**.

El desarrollo de los mercados masivos es, según yo, la más **profunda tendencia** en ingeniería software. El costo de software siempre ha sido costo de programación, y no de replica. Compartir esos costos entre incluso unos pocos usuarios disminuye radicalmente los costos por persona. Otra forma de verlo es que el uso de X cantidad de copias de un sistema software efectivamente multiplica la productividad de sus creadores X cantidad de veces. Eso es una mejora en la productividad de la disciplina y de la nación.

El problema clave es, por supuesto, es la aplicación. ¿Puedo yo usar un paquete disponible listo para ser utilizado para hacer mi trabajo? Algo sorprendente ha pasado aquí. Durante los años 50' y 60' estudio tras estudio han demostrado que los usuarios no usarían paquetes listos para utilizarse para planillas, inventarios, recibos de cuenta, etc. Los requerimientos eran muy específicos, la diferencia entre casos era muy grande. Durante los 80' encontramos tales paquetes siendo altamente requeridos y usados masivamente. ¿Qué ha cambiado?

No los paquetes. Puede que actualmente sean más generalizados y adaptables que en ese entonces, pero no es mucha la diferencia. Tampoco en las aplicaciones. De hecho las necesidades tecnológicas y de negocios actualmente son mas diversas y complicadas que las de hace 20 años atrás.

El gran cambio ha sido en el porcentaje de costos de hardware y software. En 1960 el comprador de una maquina de dos millones de dólares sentía que podía costear \$250.000 mas por un **programa de planilla personalizado**, uno que se escabulla fácilmente en el hostil ambiente social computacional. Hoy en día, el comprador de una maquina de oficina de \$50.000 no puede costear un programa de planilla personalizado, asi que adapta el procedimiento de planilla a los paquetes disponibles. Los computadores son tan comunes actualmente que las adaptaciones son acogidas rápidamente.

Existen impresionantes excepciones a mi argumento de que la generalización de paquetes de software ha cambiado poco con los años: las hojas de cálculo electrónicas y los sistemas simples de base de datos. Estas poderosas armas se prestan para muchísimos usos, algunos bastante poco ortodoxos. Hay abundantes artículos e incluso libros sobre como afrontar tareas inesperadas con la hoja de cálculo. Un gran numero de aplicaciones que actualmente habrían sido escritas por programas habituales como Cobol o Report Program Generator (programa generador de reportes) son ahora hechos con estas herramientas.

Muchos usuarios operan ahora sus computadores a diario en varias aplicaciones sin siquiera **escribir un programa**. De hecho, muchos de esos usuarios no pueden escribir nuevos programas para sus **maquinas**, pero sin embargo han logrado resolver nuevos problemas con ellos.

Yo creo que la estrategia productiva mas poderosa de software para muchas organizaciones hoy en día es la de equipar el sencillo intelecto computacional de los trabajadores que están en la línea de fuego con computadores personales y un buenos programas de escritura, dibujo, fichero y hoja de calculo y después dejarlos

libres. La misma estrategia llevada a cabo con estrategias generalizadas de matemáticas y paquetes estadísticos y algunas capacidades simples de programación también funcionara para cientos de científicos de laboratorio.

Requerimientos, refinamiento y rápida creación de prototipos

La parte más difícil en la creación de un sistema software es decidir precisamente qué construir. Ninguna otra parte del trabajo conceptual es tan difícil como establecer los detallados requerimientos técnicos, incluyendo todas las interfaces a las personas, maquinas y otros sistemas de software. Ninguna otra parte del trabajo paraliza los resultados del trabajo si se hace mal. Ninguna otra parte es mas difícil de arreglar después.

Por lo tanto, la función mas importante que el creador de software desempeña para el cliente es la extracción iterativa y refinamiento de los requerimientos del producto. La verdad es que el cliente no sabe lo que quiere. El cliente normalmente no sabe que respuestas deben ser contestadas, y casi nunca piensa en el problema en el detalle tan importante de la especificación. Incluso la simple respuesta "has que el nuevo sistema software funcione como nuestro antiguo manual de información y procesos del sistema" es de hecho demasiado simple. Uno nunca quiere exactamente eso. Sistemas software complejos son, además, cosas que actúan, se mueven, trabajan. La dinámica de esas acciones es difícil de imaginar. Así es que para planear cualquier diseño software es necesario permitir una extensa iteración entre el cliente y el diseñador como parte de la definición del sistema.

Iría más lejos y sostendría que es realmente imposible para un cliente, incluso que trabaje con ingeniería software, especificar de manera completa, precisa y correcta los exactos requerimientos de un producto software moderno antes de probar algunas versiones del producto.

Por lo tanto, uno de los esfuerzos tecnológicos más prometedores que ataca la esencia y no los accidentes del problema software es el desarrollo de aproximaciones y herramientas para la rápida creación de prototipos de sistemas ya que la creación de prototipos es parte de la iterativa especificación de requerimientos.

Un "sistema software prototipo" es uno que simula las interfaces importantes y desempeña las funciones principales del software deseado, mientras que no es necesario estar ligado por la misma velocidad, tamaño o limitaciones de costo de hardware. Los prototipos normalmente ejecutan las tareas principales de la aplicación, pero no pretenden tratar las tareas excepcionales, respondiendo correctamente a inválidas entradas de datos o abortar impecablemente. El propósito del prototipo es hacer real la estructura conceptual especificada, para que el cliente pueda probar su consistencia y utilidad.

Muchas de los actuales procedimientos software se apoyan en la suposición de que uno pueda especificar un sistema satisfactorio por adelantado, ayudando a su construcción, teniéndolo listo e instalándolo.

Grandes diseñadores:

Podemos tener buenos diseños si seguimos buenas prácticas. Las prácticas para un buen diseño pueden ser difíciles. Los programadores están entre la parte mas inteligente de la población, así que pueden aprender buenas practicas. En EEUU se esta haciendo un esfuerzo por promulgar las practicas modernas. Nuevos planes de estudio, nueva literatura, nuevas organizaciones como Software Engineering Institute, todas con el fin de aumentar el nivel de nuestra práctica.

Sin embargo, no creo que podamos subir un nivel. Ya sea que la diferencia entre diseños pobres y conceptuales yacía en el método de diseño, la diferencia entre diseños buenos y excelentes seguramente no. Grandes diseños vienen de grandes diseñadores. La construcción de un software es un proceso creativo. Metodologías sólidas pueden fortalecer y liberar la mente creativa; no puede inspirar el trabajo duro.

Las diferencias no son menores (son como las diferencias entre Salieri y Mozart). Estudio tras estudio muestra que los mejores diseñadores producen estructuras que son más rápidas, más pequeñas, más simples, más pulcras y son producidas con menor esfuerzo. Las diferencias entre el gran diseñador y el promedio son enormes.

Un poco de retrospectiva muestra que aunque muchos software buenos y útiles han sido diseñados por comités y creados como parte de proyectos (multipartes), esos sistemas software que han entusiasmado fans son el producto de una o pocas mentes diseñadoras. Consideremos Unix, APL, Pascal, Modula, la interfase Smalltalk, incluso Fortran; y contrastémoslos con Cobol, PL/I, Algol, MVS/370, y MS-DOS.

Por lo tanto, aunque yo apoyo fuertemente los esfuerzos en el desarrollo en el plan de estudios que se están desarrollando actualmente, creo que el esfuerzo más importante que podemos (preparar) es el desarrollo de métodos para aumentar grandes diseñadores.

Ninguna organización software puede ignorar este desafío. Buenos gerentes no son tan escasos como los buenos diseñadores. Grandes gerentes y grandes diseñadores son ambos muy escasos. La mayoría de las organizaciones hace considerables esfuerzos por encontrar los prospectos administrativos; no conozco ninguna que haga el mismo esfuerzo en encontrar grandes diseñadores de los que la excelencia de sus productos depende.

Mi primera propuesta es que cada organización software debe determinar y proclamar que los grandes diseñadores son tan importantes para su progreso como lo son los gerentes, y que deben ser igualmente fomentados y recompensados. No solo en cuanto al salario, también gratificaciones de reconocimiento deben ser equivalentes: tamaño de la oficina, amoblado, equipo técnico personal, fondos de viaje, equipo de apoyo.

¿Cómo aumentar a los grandes diseñadores?

- Identificar sistemáticamente a los mejores diseñadores lo más pronto posible. Los mejores no siempre son los más experimentados.

- asignar un orientador profesional para que se haga responsable por el desarrollo del prospecto.

- desarrollar y mantener un plan profesional de desarrollo para cada prospecto

- dar oportunidades de crecimiento a los diseñadores, para que interactúen y se estimulen entre sí.